

Superearn - Strategy Audit

Security Assessment

CertiK Assessed on Apr 27th, 2026





CertiK Assessed on Apr 27th, 2026

Superearn - Strategy Audit

The security assessment was prepared by CertiK.

Executive Summary

TYPES

DeFi

ECOSYSTEM

EVM Compatible

METHODS

Formal Verification, Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 04/10/2026

Final report published on 04/27/2026

Vulnerability Summary



18

Total Findings

15

Resolved

2

Multi-Sig

0

Partially Resolved

1

Acknowledged

0

Declined

2 Centralization

2 Multi-Sig



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

1 Major

1 Resolved



Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

6 Medium

6 Resolved



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

5 Minor

4 Resolved, 1 Acknowledged



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

4 Informational

4 Resolved



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | SUPEREARN - STRATEGY AUDIT

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

[Overview](#)

[External Dependencies](#)

[Privileged Functions](#)

Findings

[SSA-01 : Centralization Risk](#)

[SSA-02 : Centralized Control Of Contract Upgrade](#)

[SSA-03 : Unclearable Emergency Redemption Debt Can Permanently Block Predeposit Repayments](#)

[SSA-04 : Unreachable Shortfall Settlement Can Permanently Strand Predeposit Debt](#)

[SSA-05 : ``estimatedTotalAssets\(\)`` Can Overstate Strategy Assets While Predeposit Debt Remains Unsettled](#)

[SSA-06 : Underflow In ``getUtilizationRate\(\)`` Can Revert ``adjustPosition\(\)`` And Cause Harvest/Rebalance DoS](#)

[SSA-07 : Custody-Permission Mismatch In CustomVault Share Migration](#)

[SSA-13 : Double Pledging Of CustomVault Shares Via Preview-Based CooldownVault Debt Minting](#)

[SSA-14 : Outstanding Predeposit Debt Is Not Netted Against Idle ``want`` Holdings](#)

[SSA-08 : External Strategy Revert In ``totalAssets`` May Cause Denial Of Service](#)

[SSA-09 : Residual ERC20 Allowance Risk In Strategy Funding Flow](#)

[SSA-10 : Missing Vault Ownership Validation In Strategy Registration](#)

[SSA-11 : Unaccounted Non-Asset Tokens Can Accumulate In CustomVault](#)

[SSA-17 : Double Subtraction Of Committed Shares Causes Denial Of Service In ``premintCooldownVault``](#)

[SSA-12 : Direct Aggregation Of Share And Asset Balances In Shortfall Check](#)

[SSA-15 : Initial Strategy Binding Has No Governance Recovery Path](#)

[SSA-16 : Recorded Cooldown Is Not Enforced Before Predeposit Debt Retrieval](#)

[SSA-18 : ``CustomYearnStrategy`` Keeps A Dead Claim Path](#)

Appendix

Disclaimer

CODEBASE | SUPEREARN - STRATEGY AUDIT

Repository

<https://github.com/superearn-io/superearn-core/tree/6a9f8cd69eae036545c956dcdd6b81188ef6588b>

<https://github.com/superearn-io/superearn-core/tree/44a64d1136f848d0fece131855eddb3ed8b68000>

<https://github.com/superearn-io/superearn-core/tree/a6fb0ea4d411be047631451ffdde0c13bf584ea5>

<https://github.com/superearn-io/superearn-core/tree/91f847f23a826820e9cdab214d0ca3fc6aef58d2>

Commit

[6a9f8cd69eae036545c956dcdd6b81188ef6588b](#)

[44a64d1136f848d0fece131855eddb3ed8b68000](#)

[a6fb0ea4d411be047631451ffdde0c13bf584ea5](#)

[91f847f23a826820e9cdab214d0ca3fc6aef58d2](#)

AUDIT SCOPE | SUPEREARN - STRATEGY AUDIT

superearn-io/superearn-core

-  src/superearn/core/strategy/custom/CustomVault.sol
-  src/superearn/core/strategy/custom/CustomYearStrategy.sol
-  src/superearn/core/strategy/custom/CustomVault.sol
-  src/superearn/core/strategy/custom/CustomYearStrategy.sol
-  src/superearn/core/strategy/custom/CustomYearStrategy.sol
-  src/superearn/core/strategy/custom/CustomVault.sol
-  src/superearn/core/strategy/custom/CustomVault.sol
-  src/superearn/core/strategy/custom/CustomYearStrategy.sol

APPROACH & METHODS | SUPEREARN - STRATEGY AUDIT

This audit was conducted for Superearn to evaluate the security and correctness of the smart contracts associated with the Superearn - Strategy Audit project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Formal Verification, Manual Review, and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | SUPEREARN - STRATEGY AUDIT

Overview

SuperEarn - Strategy Audit covers the custom strategy stack formed by `CustomVault` and `CustomYearnStrategy`.

`CustomVault` is an ERC4626-style vault that serves as the strategy's external position layer and can allocate funds to governance-approved custom strategies. `CustomYearnStrategy` is a Yearn-compatible strategy that deposits into `CustomVault`, holds `CustomVault` shares as its external position, and manages the related predeposit debt and repayment flow.

External Dependencies

The reviewed contracts interact with external libraries, vault components, tokens, and strategy integrations. For this review, these dependencies are treated as out of scope and assumed to behave correctly. In practice, failures, upgrades, or compromise of third-party components may affect accounting, access control, liveness, or asset safety.

The following libraries/contracts are considered third-party dependencies:

- `@openzeppelin/contracts-upgradeable/`
- `@openzeppelin/contracts/`
- `@yearn-vaults/`
- `ICooldownVault`
- The Yearn vault integrating `CustomYearnStrategy`
- `ICustomStrategy` implementations registered in `CustomVault`
- `TransparentUpgradeableProxy` and the associated upgrade admin for `CustomVault`

We assume these contracts, tokens, libraries, and privileged external components are valid, correctly configured, and non-malicious.

Privileged Functions

In the **SuperEarn - Strategy Audit** project, privileged roles are adopted to ensure the dynamic runtime updates of the project, which are specified in the **Centralization Risks** finding.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime requirements to best serve the community. It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of a `TimeLock` contract.

FINDINGS | SUPEREARN - STRATEGY AUDIT



18
Total Findings

0
Critical

2
Centralization

1
Major

6
Medium

5
Minor

4
Informational

This report has been prepared for Superearn to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 18 issues were identified. Leveraging a combination of Formal Verification, Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
SSA-01	Centralization Risk	Centralization	Centralization	4/5 Multi-Sig
SSA-02	Centralized Control Of Contract Upgrade	Centralization	Centralization	4/5 Multi-Sig
SSA-03	Unclearable Emergency Redemption Debt Can Permanently Block Predeposit Repayments	Logical Issue	Major	Resolved
SSA-04	Unreachable Shortfall Settlement Can Permanently Strand Predeposit Debt	Inconsistency	Medium	Resolved
SSA-05	<code>estimatedTotalAssets()</code> Can Overstate Strategy Assets While Predeposit Debt Remains Unsettled	Logical Issue	Medium	Resolved
SSA-06	Underflow In <code>getUtilizationRate()</code> Can Revert <code>adjustPosition()</code> And Cause Harvest/Rebalance DoS	Logical Issue	Medium	Resolved
SSA-07	Custody-Permission Mismatch In CustomVault Share Migration	Logical Issue	Medium	Resolved
SSA-13	Double Pledging Of CustomVault Shares Via Preview-Based CooldownVault Debt Minting	Financial Manipulation	Medium	Resolved
SSA-14	Outstanding Predeposit Debt Is Not Netted Against Idle <code>want</code> Holdings	Logical Issue	Medium	Resolved
SSA-08	External Strategy Revert In <code>totalAssets</code> May Cause Denial Of Service	Volatile Code	Minor	Acknowledged

ID	Title	Category	Severity	Status
SSA-09	Residual ERC20 Allowance Risk In Strategy Funding Flow	Volatile Code	Minor	● Resolved
SSA-10	Missing Vault Ownership Validation In Strategy Registration	Logical Issue	Minor	● Resolved
SSA-11	Unaccounted Non-Asset Tokens Can Accumulate In CustomVault	Volatile Code	Minor	● Resolved
SSA-17	Double Subtraction Of Committed Shares Causes Denial Of Service In <code>premintCooldownVault</code>	Logical Issue	Minor	● Resolved
SSA-12	Direct Aggregation Of Share And Asset Balances In Shortfall Check	Volatile Code	Informational	● Resolved
SSA-15	Initial Strategy Binding Has No Governance Recovery Path	Coding Style	Informational	● Resolved
SSA-16	Recorded Cooldown Is Not Enforced Before Predeposit Debt Retrieval	Logical Issue	Informational	● Resolved
SSA-18	<code>CustomYearnStrategy</code> Keeps A Dead Claim Path	Inconsistency	Informational	● Resolved

SSA-01 | Centralization Risk

Category	Severity	Location	Status
Centralization	● Centralization	src/superearn/core/strategy/BaseCooldownStrategy.sol (init): 419, 447, 456, 469, 490, 504, 514; src/superearn/core/strategy/custom/CustomVault.sol (init): 227, 254, 288, 313, 340, 350; src/superearn/core/strategy/custom/CustomYearnStrategy.sol (init): 313, 359, 359, 540, 553, 562; src/yearn-vaults/BaseStrategy.sol (init): 215, 220, 251, 265, 285, 302, 316, 331, 344, 357, 612, 711, 757	● 4/5 Multi-Sig

Description

In the contract `CustomVault`, the role `governance` has authority over the following functions:

- `addCustomStrategy()`
- `removeCustomStrategy()`
- `depositToCustomStrategy()`
- `withdrawFromCustomStrategy()`
- `setOperator()`
- `transferGovernance()`
- `setCustomYearnStrategy()`

Any compromise to the `governance` account may allow an attacker to arbitrarily register/remove strategies, move vault assets into or out of strategies (directly via governance's `onlyOperator` bypass), delegate operator permissions, and transfer governance control.

In the contract `CustomVault`, the role `operator` has authority over the following functions:

- `depositToCustomStrategy()`
- `withdrawFromCustomStrategy()`

Any compromise to an `operator` account may allow an attacker to rebalance or move vault funds across registered custom strategies in an unauthorized manner, which can materially affect fund safety and liquidity.

In the contract `CustomVault`, the role `pendingGovernance` has authority over the following functions:

- `acceptGovernance()`

Any compromise to the `pendingGovernance` account may allow an attacker to finalize governance takeover and then exercise full governance-level control over strategy and operator management.

In the contract `CustomVault`, the role `customYearnStrategy` has authority over the following functions:

- `deposit()`
- `mint()`
- `withdraw()`
- `redeem()`
- `migrateCustomYearnStrategy()`

Any compromise to the `customYearnStrategy` account may allow an attacker to directly control the vault's core ERC4626 asset flow, including depositing assets, minting shares, changing the custom yearn strategy address, and withdrawing or redeeming vault assets, which can materially affect asset custody, share accounting, and liquidity management.

In the contract `CustomYearnStrategy`, the role `governance` has authority over the following functions:

- `setHealthCheck()`
- `setDoHealthCheck()`
- `setStrategist()`
- `setKeeper()`
- `setRewards()`
- `setMinReportDelay()`
- `setMaxReportDelay()`
- `setCreditThreshold()`
- `setForceHarvestTriggerOnce()`
- `setBaseFeeOracle()`
- `setMetadataURI()`
- `tend()`
- `harvest()`
- `setEmergencyExit()`
- `sweep()`
- `emergencyRedeem()`
- `emergencyRepayRemainingPredepositDebt()`
- `setShortfallTolerance()`
- `submitExecution()`
- `acceptExecution()`
- `cancelExecution()`
- `setAllowedTarget()`
- `withdrawFromCustomVault()`
- `setCooldownPeriod()`
- `setReserveRatio()`

- `emergencyClaim()`

Any compromise to the `governance` account may allow an attacker to change core strategy operators and runtime parameters, trigger tend/harvest and emergency-exit flows, sweep non-protected tokens, manually redeem `CustomVault` shares, repay recorded shortfall, and allowlist plus execute arbitrary external calls (with a configurable delay that defaults to zero). Note that `emergencyClaim()` is governance-only but intentionally unsupported in this implementation because it always reverts.

In the contract `CustomYearnStrategy`, the role `strategist` has authority over the following functions:

- `setStrategist()`
- `setKeeper()`
- `setRewards()`
- `setMinReportDelay()`
- `setMaxReportDelay()`
- `setMetadataURI()`
- `tend()`
- `harvest()`
- `setEmergencyExit()`
- `cancelExecution()`
- `withdrawFromCustomVault()`

Any compromise to the `strategist` account may allow an attacker to reassign key operators, alter reporting cadence and execution metadata, run strategy lifecycle operations, trigger emergency exit, cancel pending timelock actions, and manually redeem strategy shares from `CustomVault`.

In the contract `CustomYearnStrategy`, the role `keeper` has authority over the following functions:

- `tend()`
- `harvest()`
- `withdrawFromCustomVault()`

Any compromise to the `keeper` account may allow an attacker to repeatedly trigger position adjustments/harvest cycles and manually withdraw from `CustomVault`, potentially causing harmful timing, liquidity disruption, or adverse execution conditions.

In the contract `CustomYearnStrategy`, the role `vault.management` has authority over the following functions:

- `setHealthCheck()`
- `setDoHealthCheck()`
- `setCreditThreshold()`
- `setForceHarvestTriggerOnce()`
- `setBaseFeeOracle()`
- `tend()`

- `harvest()`
- `setEmergencyExit()`
- `withdrawFromCustomVault()`

Any compromise to the `vault.management` account may allow an attacker to manipulate health-check and harvest-trigger configuration, force tend/harvest operations, place the strategy into emergency-exit mode, and manually redeem `CustomVault` shares into underlying USDT.

In the contract `CustomYearnStrategy`, the role `vault.guardian` has authority over the following functions:

- `tend()`
- `harvest()`
- `setEmergencyExit()`
- `withdrawFromCustomVault()`

Any compromise to the `vault.guardian` account may allow an attacker to force tend/harvest operations, activate emergency exit, and manually redeem `CustomVault` shares into underlying USDT, which can disrupt liquidity management and strategy behavior.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (2/3, 4/5) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (2/3, 4/5) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.

- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

I Alleviation

[Superearn, 04/23/2026]:

I Kaia Mainnet

Key Addresses

Name	Address
KAIA_GOVERNANCE_MULTISIG	0x694B81Db6E16c75B6B9EF9F1b09a

Name	Address
	a3FD1F1d5f05
KAIA_MANAGEMENT_MULTISIG	0xAf65b84B306b4a39EcCc3c915A9143956BC935C8
KAIA_REVENUE_MULTISIG	0xB1Bd9B1F1F954b916104D9b000B57BfC717B666C
KAIA_LIGHT_KEEPER	0x8c82B2feC291a43e41aA87669eaEf01F4efaA3B2
KAIA_CROSSCHAIN_KEEPER	0x40FB0F9084828ADBc3dcd71840eA545BF243cD0F
KAIA_HEALTH_CHECK	0x8410eBD5B77F17751F1044D3D3b6BDacE00DE7b8
KAIA_PROXY_ADMIN	0x9Ef977E4521ca735a870cBFA8bA225F558866B4B
KAIA_YVAULT (USDT)	0x2e4e573D86c70688cD97D76bc5DDc1Bb265bF5D6
KAIA_COOLDOWN_VAULT	0x4E4654cE4Ca7ff0ba66a0A4a588A4bd55A6f9A33
KAIA_YEAR_N_VAULT_MANAGER	0x763CD200971c41a84552d3D691D60764E256c163
Deployer EOA	0x4C9abFe68A896750f5c6CD7129cc0F803528DB27

[CertiK, 04/23/2026]:

Based on the on-chain deployment information provided by the team, the following configuration has been verified.

CustomYearnStrategy

The `CustomYearnStrategy` contract is deployed at:

(<https://kaiascan.io/address/0x723d3422788f47f5dae153515a3c277293dbd8f3>)

The associated **Yearn Vault** is deployed at:

(<https://kaiascan.io/address/0x2e4e573D86c70688cD97D76bc5DDc1Bb265bF5D6>)

governance

The **governance address** is:

0x694B81Db6E16c75B6B9EF9F1b09aa3FD1F1d5f05

This address is configured as a **multisignature wallet** with a **4-of-5 signature threshold**, the multisig signers are:

- 0xbFA4A3430774376CAB83A81FaE301f7b05d90f34
- 0x449b1cF9632454D60F1B55125F10B475a45c86c6
- 0x9A3cE8ca1372114Ec73Ad537F1C6B9594d9FFBbf
- 0x7C0d7Cc8D87e78B8167344C13F06Ca9f381E07F8
- 0xD7832AD89CBac201F2Dc33e41B666f878e882A4F

strategist

The **strategist address** is:

0xAf65b84B306b4a39EcCc3c915A9143956BC935C8

This address is configured as a **multisignature wallet** with a **2-of-3 signature threshold**, the multisig signers are:

- 0x19C7Cf8c28df30963B6436a22d4eeE0cd99585F7
- 0x9Fa3Af580e4d27Bf11F49F6c582d3dC40140d6A4
- 0x374f7713Ab5A50cE7E779BbF71B803f311352bF3

keeper

The keeper contract is deployed at: 0x8c82B2feC291a43e41aA87669eaEf01F4efaA3B2.

vault.management

The management contract is deployed at: 0x763CD200971c41a84552d3D691D60764E256c163.

vault.guardian

The **guardian address** is:

0xAf65b84B306b4a39EcCc3c915A9143956BC935C8

This address is configured as a **multisignature wallet** with a **2-of-3 signature threshold**, the multisig signers are:

- 0x19C7Cf8c28df30963B6436a22d4eeE0cd99585F7
- 0x9Fa3Af580e4d27Bf11F49F6c582d3dC40140d6A4
- 0x374f7713Ab5A50cE7E779BbF71B803f311352bF3

CustomVault

The proxy contract is deployed at: 0x7876a2faf6aad1f6f8e47ad612d9472a4821dfda.

The implementation contract is deployed at: 0x90a2b553845bdc0f4c43554ed306d09fb632f259.

governance

The **governance address** is:

0x694B81Db6E16c75B6B9EF9F1b09aa3FD1F1d5f05

This address is configured as a **multisignature wallet** with a **4-of-5 signature threshold**, the multisig signers are:

- 0xbFA4A3430774376CAB83A81FaE301f7b05d90f34
- 0x449b1cF9632454D60F1B55125F10B475a45c86c6
- 0x9A3cE8ca1372114Ec73Ad537F1C6B9594d9FFBbf
- 0x7C0d7Cc8D87e78B8167344C13F06Ca9f381E07F8
- 0xD7832AD89CBac201F2Dc33e41B666f878e882A4F

operator

The operator contracts are listed below:

- [0x40FB0F9084828ADBC3dcd71840eA545BF243cD0F](#)
- [0x8c82B2feC291a43e41aA87669eaEf01F4efaA3B2](#).

customYearnStrategy

The custom Yearn strategy contract is deployed at: [0x723d3422788f47f5DaE153515A3C277293dbd8f3](#).

pendingGovernance

The pending governance address is currently set to the zero address: `0x0000000000000000000000000000000000000000000000000000000000000000`.

As of this review, based on on-chain read-only queries and the [deployment transaction](#), we verified that the governance address [0x694B81Db6E16c75B6B9EF9F1b09aa3FD1F1d5f05](#) is a Safe proxy deployed via the [SafeProxyFactory](#). Its `masterCopy()` points to the [SafeL2 singleton](#), and no Safe modules are currently enabled.

Similarly, the strategist/guardian address [0xAf65b84B306b4a39EcCc3c915A9143956BC935C8](#) was deployed in transaction [0x856143072cce843295a358052ed2719d2981ebdc1417319bde964fbc783f4ff5](#) via the [SafeProxyFactory](#). Its `masterCopy()` points to the [SafeL2 singleton](#), and `getModulesPaginated()` indicates that no Safe modules are currently enabled.

While this strategy has indeed reduced the risk, it's crucial to note that it has not completely eliminated it. CertiK strongly encourages the project team to periodically revisit the private key security management of all above-listed addresses.

SSA-02 | Centralized Control Of Contract Upgrade

Category	Severity	Location	Status
Centralization	● Centralization	src/superearn/core/strategy/custom/CustomVault.sol (init): 21~22	● 4/5 Multi-Sig

Description

The following contract is upgradeable and is intended to be used behind a proxy, with upgrades controlled by the proxy admin:

- CustomVault

CustomVault is implemented using OpenZeppelin's Initializable pattern and is intended to be deployed behind a TransparentUpgradeableProxy. Accordingly, the proxy admin, or the owner of the ProxyAdmin contract if a ProxyAdmin is used, can upgrade the CustomVault implementation at any time.

If this upgrade authority is compromised or acts maliciously, an attacker could replace the CustomVault implementation with arbitrary logic and execute malicious functionality through the proxy, potentially bypassing access controls, altering accounting, or putting vault assets at risk.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (2/3, 4/5) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (2/3, 4/5) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

Alleviation

[Superearn, 04/23/2026]: The team acknowledged the issue and adopted the multisign solution to ensure the private key management process at the current stage. The `proxyAdmin` of `CustomVault` contract `proxy` has transferred the ownership to a Gnosis Safe contract with 4/5 signers in the sensitive function signing process.

- Grant Role transaction hash for Gnosis Safe:
0xd3ec0d4289536c6b38518f8dc6ee04b9f8db7349b93dd0526f7196ae2020f68d, Gnosis safe contract address:

0x694B81Db6E16c75B6B9EF9F1b09aa3FD1F1d5f05

- The 5 multisign addresses:
 1. EOA:0xbFA4A3430774376CAB83A81FaE301f7b05d90f34
 2. EOA:0x449b1cF9632454D60F1B55125F10B475a45c86c6
 3. EOA:0x9A3cE8ca1372114Ec73Ad537F1C6B9594d9FFBbf
 4. EOA:0x7C0d7Cc8D87e78B8167344C13F06Ca9f381E07F8
 5. EOA:0xD7832AD89CBac201F2Dc33e41B666f878e882A4F

[CertiK, 04/23/2026]: While this strategy has indeed reduced the risk, it's crucial to note that it has not completely eliminated it. CertiK strongly encourages the project team to periodically revisit the private key security management of all above-listed addresses.

SSA-03 | Unclearable Emergency Redemption Debt Can Permanently Block Predeposit Repayments

Category	Severity	Location	Status
Logical Issue	● Major	src/superearn/core/strategy/custom/CustomYearnStrategy.sol (init): 311	● Resolved

Description

`CustomYearnStrategy` inherits `emergencyRedeem()` from `BaseCooldownStrategy`. That flow calls `requestRedeem()`, but in this contract `requestRedeem()` does not withdraw from `CustomVault`; it only creates a `DebtRecord` and increases `accDebtAmount`. Because the debt created during `emergencyRedeem()` is not tied to any `predepositId`, it cannot be cleared by `repayPredepositDebt()`, and `emergencyClaim()` is overridden to revert. This can leave a permanent phantom debt that increases `reservedForPriorDebts` and can block or distort later real predeposit repayments.

The issue comes from how `CustomYearnStrategy` uses inherited emergency logic from `BaseCooldownStrategy`.

- In `CustomYearnStrategy`, `emergencyRedeem()` is inherited from `BaseCooldownStrategy` and calls `requestRedeem()`. However, `CustomYearnStrategy.requestRedeem()` does not redeem assets from `CustomVault`. Instead, it only creates a `DebtRecord` and increments `accDebtAmount`.
- The cumulative values from those `DebtRecord` entries and `accDebtAmount` are later used by `predepositDebtRetrievable()` and `repayPredepositDebt()` to determine how much USDT must be reserved for prior debts.
- The debt created through `emergencyRedeem()` is not associated with any `predepositId`. Because of that, `repayPredepositDebt()` has no way to match and clear it.
- `CustomYearnStrategy` also overrides `emergencyClaim()` to revert. This removes the emergency path that might otherwise resolve the debt created by `emergencyRedeem()`.

Together, these behaviors allow `emergencyRedeem()` to create debt that is counted in accounting but cannot be settled.

As `emergencyRedeem()` can create a permanent phantom debt in `CustomYearnStrategy`, for any later real predeposit mapped to a higher `redeemIndex`, both `predepositDebtRetrievable()` and `repayPredepositDebt()` reserve `reservedForPriorDebts` using `accDebtAmount[redeemIndex-1] - accRepaidDebtAmount`, so the un-clearable emergency debt can make debts appear unretrievable / repayments revert.

Recommendation

Do not use the inherited emergency redemption flow in `CustomYearnStrategy` unless it is made compatible with the strategy's debt-accounting model. Ensure that any emergency path either redeems assets immediately from `CustomVault` without creating a `DebtRecord`, or creates debt that is fully traceable and removable through the same lifecycle as normal predeposit debt, including association with a valid `predepositId` or an equivalent clearable identifier.

Align the emergency functions so that `emergencyRedeem()`, `requestRedeem()`, `repayPredepositDebt()`, and `emergencyClaim()` operate on a consistent state model. In particular, prevent emergency-created debt from contributing to `accDebtAmount` and `reservedForPriorDebts` unless there is a guaranteed mechanism to settle or clear it.

If emergency redemption is not intended to be supported by this strategy, explicitly override `emergencyRedeem()` to revert, rather than inheriting behavior that produces unresolvable accounting state.

■ Alleviation

[Superearn, 04/12/2026]: The team heeded the advice and resolved the issue by overriding `emergencyRedeem()` to revert in commit [7f1a6961be9e785445556f616acb62cf3b4ff122](#).

SSA-04 | Unreachable Shortfall Settlement Can Permanently Strand Predeposit Debt

Category	Severity	Location	Status
Inconsistency	● Medium	src/superearn/core/strategy/custom/CustomYearnStrategy.sol (init): 265, 311	● Resolved

Description

`CooldownVault.retrieveDebt()` only proceeds when `strategy.predepositDebtRetrievable(predepositId)` returns `true`. In `CustomYearnStrategy`, `predepositDebtRetrievable(uint256 predepositId)` returns `true` only when the strategy can fully repay the debt after reserving liquidity for earlier debts. However, `repayPredepositDebt(uint256 predepositId)` also contains a separate shortfall path for insolvency or depletion cases. Because the call is blocked before `repayPredepositDebt()` runs, that shortfall path is unreachable, which can leave bad predeposit debt stuck and unfinalized.

The main issue is a call-flow mismatch between `CooldownVault.retrieveDebt()` and `CustomYearnStrategy`.

`CooldownVault.retrieveDebt()` first checks `strategy.predepositDebtRetrievable(predepositId)`. It only calls `repayPredepositDebt(predepositId)` when that check returns `true`.

In `CustomYearnStrategy.predepositDebtRetrievable(uint256 predepositId)`, the function:

- computes `reservedForPriorDebts` from `accDebtAmount[redeemIndex - 1]` and `accRepaidDebtAmount`
- computes `available` from `externalUnderlyingToken.balanceOf(address(this)) - reservedForPriorDebts` when positive
- returns only `available >= record.usdtAmount`

This means `predepositDebtRetrievable()` recognizes only the full repayment case.

In contrast, `CustomYearnStrategy.repayPredepositDebt(uint256 predepositId)` explicitly supports two outcomes:

- full repayment when `available >= predepositDebt`
- shortfall settlement when `available < predepositDebt`, if:
 - `customVault.totalAssets() <= DUST_THRESHOLD` or `externalShareToken.balanceOf(address(this)) == 0`
 - and `usdtBalance + want.balanceOf(address(this)) < predepositDebt`

In that shortfall branch, the function is designed to:

- set `repayAmount = available`
- increase `remainingPredepositDebt` by `predepositDebt - available`

- mark `record.isClaimed = true`
- increase `accRepaidDebtAmount` by the full `record.usdtAmount`

Because `CooldownVault.retrieveDebt()` gates execution on `predepositDebtRetrievable(predepositId)`, and `predepositDebtRetrievable()` returns `false` whenever `available < record.usdtAmount`, the shortfall branch inside `repayPredepositDebt()` cannot be reached through the normal call path. The exact state that should trigger bad-debt crystallization is filtered out before repayment logic executes.

In a loss event, the intended shortfall handling cannot be completed.

`CooldownVault.retrieveDebt()` reverts with `DebtUnretrievable()` instead of allowing `repayPredepositDebt()` to settle the debt partially.

Recommendation

Align the eligibility check in `CooldownVault.retrieveDebt()` with the actual settlement logic in `CustomYearnStrategy.repayPredepositDebt()`. `predepositDebtRetrievable(uint256 predepositId)` should return `true` not only for full repayment cases, but also when the strategy is in a terminal shortfall state where partial settlement and bad-debt crystallization are intended.

Additionally, verify that once a shortfall state is reached, the debt can always be finalized by updating claim status, recording the unpaid remainder in `remainingPredepositDebt`, and advancing aggregate debt accounting to prevent the entry from remaining permanently pending.

Alleviation

[Superearn, 04/12/2026]: The team heeded the advice and resolved the issue by allowing shortfall settlement when `CustomVault` is depleted and strategy cannot cover the debt — matches the shortfall branch in `repayPredepositDebt()` in commit [29ea519b8d5c800ca47c854dadd1d64b22b187d7](#).

SSA-05 `estimatedTotalAssets()` Can Overstate Strategy Assets While Predeposit Debt Remains Unsettled

Category	Severity	Location	Status
Logical Issue	Medium	src/superearn/core/strategy/BaseCooldownStrategy.sol (init): 571; src/superearn/core/strategy/custom/CustomYearnStrategy.sol (init): 165	Resolved

Description

`BaseCooldownStrategy.estimatedTotalAssets()` can overstate strategy assets while predeposit debt remains unsettled.

When `requestRedeem()` is called, through the call-chain `CustomYearnStrategy.liquidatePosition()` -> `BaseCooldownStrategy.premintCooldownVault()` -> `CustomYearnStrategy.requestRedeem()`, it only records an internal debt entry. It does not burn/remove the `CustomVault` shares from the strategy's balance.

Inside the `BaseCooldownStrategy.premintCooldownVault()`, the `cooldownVault.predeposit()` then mints new `CooldownVault` shares to the strategy for immediate liquidity. Now the same underlying value is counted twice in `estimatedTotalAssets()`:

- Once as `externalShareToken.balanceOf(address(this))` (`CustomVault` shares still held)
- Once as `want.balanceOf(address(this))` (newly minted `CooldownVault` shares)

The root cause is that the `CustomYearnStrategy.requestRedeem()` function records predeposit debt but does not burn or otherwise remove the corresponding `externalShareToken` position from strategy accounting. As a result, the value backing that debt continues to be included through `previewRedeem(externalShareToken.balanceOf(address(this)))` even before any manual vault withdrawal occurs.

While predeposit debt is outstanding, harvest accounting can treat overlapped assets as available value and miscompute `profit` or `debtPayment` in `prepareReturn()`. This can distort vault accounting during intermediate states where debt has been recorded but not yet repaid.

Recommendation

Adjust strategy accounting so that assets backing outstanding predeposit debt are excluded from `estimatedTotalAssets()` until the corresponding debt is repaid or otherwise settled. Specifically, ensure `prepareReturn()` does not treat debt-backed `externalShareToken`, redeemed `externalUnderlyingToken`, or other intermediate asset representations as additional available assets while the related predeposit liability remains open.

Alleviation

[Superearn, 04/12/2026]: The team heeded the advice and resolved the issue by excluding predeposit debt in commit [003129fbe6d6bcc7dc5ba08f6f7bc702a496c48b](#).

SSA-06 Underflow In `getUtilizationRate()` Can Revert `adjustPosition()` And Cause Harvest/Rebalance DoS

Category	Severity	Location	Status
Logical Issue	● Medium	src/superearn/core/strategy/BaseCooldownStrategy.sol (init): 555; src/superearn/core/strategy/custom/CustomYearnStrategy.sol (init): 498	● Resolved

Description

The `adjustPosition()` function emits `AdjustPosition` using `this.getUtilizationRate()` at the end of execution. In `getUtilizationRate()`, the expression `estimatedTotalAssets() - want.balanceOf(address(this))` is not floored at zero. If `estimatedTotalAssets()` is positive but smaller than `want.balanceOf(address(this))`, this subtraction underflows and the call reverts. As a result, `adjustPosition()` can complete its logic but still revert at the final event emission, causing a denial of service for harvests or rebalances during stress.

The root cause is in `getUtilizationRate()`. It computes `estimatedTotalAssets() - want.balanceOf(address(this))` without handling the case where `want.balanceOf(address(this))` is greater than `estimatedTotalAssets()`. In that state, the subtraction underflows and reverts.

Impact

A keeper may be unable to call `adjustPosition()` successfully, because the transaction reverts only at the final event emission.

This creates a *denial of service* for harvest or rebalance operations.

Recommendation

Floor the utilization numerator at zero in `getUtilizationRate()` so the calculation cannot underflow when `want.balanceOf(address(this))` exceeds `estimatedTotalAssets()`. More broadly, treat idle or temporarily unavailable assets consistently across `estimatedTotalAssets()` and utilization accounting so the metric cannot enter an invalid state during shortfall or partial redemption conditions.

Alleviation

[Superearn, 04/11/2026]: The team heeded the advice and resolved the issue by flooring numerator at zero in commit [6b88a629804f5d185a53c705a4392d489a73fa68](#).

SSA-07 | Custody-Permission Mismatch In CustomVault Share Migration

Category	Severity	Location	Status
Logical Issue	● Medium	src/superearn/core/strategy/custom/CustomVault.sol (init): 143~152, 178~179, 193~194; src/superearn/core/strategy/custom/CustomYearnStrategy.sol (init): 506~510	● Resolved

Description

`CustomVault` restricts the execution of `deposit`, `mint`, `withdraw`, and `redeem` functions to a single address specified by `customYearnStrategy`.

During normal operation, the `CustomYearnStrategy` contract calls `customVault.deposit(assets, address(this))` within its `requestDeposit` function, minting and holding ERC4626 shares in the strategy's own address.

```
// CustomYearnStrategy: deposit path mints shares to the strategy itself
try customVault.deposit(assets, address(this)) returns (uint256 sharesReceived) {
    ...
}
```

During a migration, `prepareMigration` transfers the share tokens from the old strategy to the new one, but the authorization variable within `CustomVault` that defines which strategy is permitted to interact with these functions is not updated at the same time as share custody is transferred.

```
// CustomYearnStrategy: migration transfers share custody
uint256 externalShareBal = externalShareToken.balanceOf(address(this));
if (externalShareBal > 0) {
    externalShareToken.safeTransfer(_newStrategy, externalShareBal);
}
```

```
// CustomVault: execution authority pinned to one strategy
modifier onlyCustomYearnStrategy() {
    if (msg.sender != customYearnStrategy) revert NotAuthorized();
    _;
}
```

This results in a scenario where, post-migration, the new strategy holds the necessary shares but does not have permission to interact with the vault for critical functions like `redeem` or `withdraw`. Meanwhile, the old strategy may still retain the authority but no longer controls the shares. This misalignment between share ownership and execution rights can disrupt accounting and operability, requiring manual intervention to resolve in a live environment.

Recommendation

The migration process should ensure that share transfers are tightly coupled with the corresponding execution authority update within `CustomVault` .

■ Alleviation

[Superearn, 04/13/2026]: The team heeded the advice and resolved the issue by atomically handing off strategy authority during migration in commit [bab020e6aff35371b3c08aa53fade39b374e9909](#).

SSA-13 Double Pledging Of CustomVault Shares Via Preview-Based CooldownVault Debt Minting

Category	Severity	Location	Status
Financial Manipulation	● Medium	src/superearn/core/strategy/BaseCooldownStrategy.sol (0413-44a6): 296; src/superearn/core/strategy/custom/CustomYearnStrategy.sol (0413-44a6): 166, 477	● Resolved

Description

The `CustomYearnStrategy` can create `CooldownVault` debt and immediate liquidity from `CustomVault` shares that were only *previewed*, not actually redeemed or reserved. The `requestRedeem()` records a fixed `DebtRecord` using `customVault.previewRedeem()` but leaves the same shares in `externalShareToken.balanceOf(address(this))`. Because later capacity checks still count those shares as available, the strategy can reuse the same position to back multiple pre-mints.

Though the new version(commit:44a64d) `estimatedTotalAssets()` excluded the `outstandingDebt` to mitigate the issue, the `premintCooldownVault()` sizes new minting from the unchanged gross external share balance:

```

281     function premintCooldownVault(uint256 sharesNeeded)
282         ...
283
284     // When requesting based on shares, the actual amount received is often less than
285     // expected.
286
287     // Receiving slightly more than this is not an issue, so we add a buffer of 10
288     // amount.
289
290     uint256 _needUsdt = cooldownVault.previewMint(sharesNeeded);
291     uint256 _needSusdt = previewWithdraw(_needUsdt + AMOUNT_TO_SHARE_BUFFER
292 );
293     uint256 _susdtBalance = externalShareToken.balanceOf(address(this));
294     //<@
295     uint256 shares = Math.min(_needSusdt, _susdtBalance);

```

And, the `liquidatePosition(uint256 _amountNeeded)` directly calls the `premintCooldownVault(_amountNeeded - availableWithoutReserve)` to get liquidity.

As a result, the same shares remain in `externalShareToken.balanceOf(address(this))` and are still treated as available for the `liquidatePosition(uint256 _amountNeeded)` scenario.

Scenario

Repeated withdrawal before manual settlement

- The strategy holds `CustomVault` shares in `externalShareToken.balanceOf(address(this))`.
- A withdrawal reaches `liquidatePosition(uint256 _amountNeeded)`, which calls `premintCooldownVault(...)`.
- `premintCooldownVault()` calls `requestRedeem(shares)`, which stores `DebtRecord.usdtAmount = customVault.previewRedeem(shares)` but does not redeem or reserve the shares.
- Before a keeper or manager calls `withdrawFromCustomVault()`, another withdrawal happens.
- Because `externalShareToken.balanceOf(address(this))` is unchanged, `premintCooldownVault()` treats the same shares as available again and creates more debt against them.
- When settlement finally happens, actual redeemed assets may be insufficient to cover all recorded debt.

I Recommendation

Track and enforce an explicit distinction between gross `CustomVault` shares and shares already committed to outstanding `DebtRecord` s. Any shares referenced by `requestRedeem(uint256 shares)` should be treated as encumbered immediately and excluded from future capacity checks, e.g., `premintCooldownVault(uint256 sharesNeeded)`.

I Alleviation

[Superearn, 04/16/2026]: The team heeded the advice and resolved the issue by preventing double-premint in commit [a6fb0ea4d411be047631451ffdde0c13bf584ea5](#).

SSA-14 Outstanding Predeposit Debt Is Not Netted Against Idle want Holdings

Category	Severity	Location	Status
Logical Issue	● Medium	src/superearn/core/strategy/custom/CustomYearnStrategy.sol (0413-44a6): 396	● Resolved

Description

In `CustomYearnStrategy.estimatedTotalAssets()`, outstanding predeposit debt from `cooldownVault.strategyDebtOutstanding(address(this))` is netted only against the external USDT/CustomVault position, while `want.balanceOf(address(this))` is always added in full.

When external assets are worth less than the outstanding debt, the uncovered liability is ignored and zeroed instead of reducing idle `want`, so reported assets are overstated.

```

396     function estimatedTotalAssets() public view virtual override returns (
uint256) {
397         uint256 pendingInvested = want.balanceOf(address(this));
398
399         uint256 underlyingToPreview = externalUnderlyingToken.balanceOf(address
(this));
400         underlyingToPreview += previewRedeem(externalShareToken.balanceOf(
address(this)));
401
402
403         // Exclude USDT obligated to CooldownVault – these funds back `pendingInvested`
404         // (the preminted CooldownVault shares) and must not count as free assets.
405         uint256 outstandingDebt = cooldownVault.strategyDebtOutstanding(address
(this));
406         underlyingToPreview = underlyingToPreview > outstandingDebt ?
underlyingToPreview - outstandingDebt : 0; //<@
407
408         uint256 previewInWant = cooldownVault.previewDeposit(
underlyingToPreview);
409         return pendingInvested + previewInWant;
410     }

```

The function computes `pendingInvested = want.balanceOf(address(this))`, values the external position as `externalUnderlyingToken.balanceOf(address(this)) + previewRedeem(externalShareToken.balanceOf(address(this)))`, subtracts `outstandingDebt` only from that external bucket with `underlyingToPreview = underlyingToPreview > outstandingDebt ? underlyingToPreview - outstandingDebt : 0`, and then returns `pendingInvested + cooldownVault.previewDeposit(underlyingToPreview)`.

Because that subtraction is floored at zero before `pendingInvested` is considered, any deficit where external assets are below `strategyDebtOutstanding` is discarded rather than propagated into idle `want`.

As a result, it can suppress or delay recognition of real losses in `prepareReturn()`. A direct profit-manufacturing payout through `prepareReturn()` is not established by the described scenario.

Recommendation

Net outstanding predeposit debt against the strategy's total assets, not only the external USDT/CustomVault position. In `estimatedTotalAssets()`, account for `cooldownVault.strategyDebtOutstanding(address(this))` across both the external position and idle `want`.

Alleviation

[Superearn, 04/16/2026]: The team heeded the advice and resolved the issue by propagating excess debt into `pendingInvested` in `estimatedTotalAssets` in commit [fdcd0710805eef7dc1286e566b02f04c35cb970c](#).

SSA-08 External Strategy Revert In `totalAssets` May Cause Denial Of Service

Category	Severity	Location	Status
Volatile Code	Minor	src/superearn/core/strategy/custom/CustomVault.sol (init): 212~213	Acknowledged

Description

`CustomVault.totalAssets()` is designed to aggregate idle vault assets plus the assets reported by each registered custom strategy, which makes it a critical accounting dependency for pricing and repayment paths. The current implementation performs direct external calls in a loop without fault isolation, so a single strategy reverting in `totalAssets()` can revert the entire vault-level valuation flow.

This risk is amplified because `CustomYearnStrategy.repayPredepositDebt()` also depends on `customVault.totalAssets()` in its shortfall branch, allowing one faulty or malicious strategy to degrade system-wide liveness during debt settlement windows. The core risk source is fail-open trust in external strategy view calls on a critical path, as illustrated by the pattern below:

```
for (uint256 i = 0; i < length; i++) {
    total += ICustomStrategy(customStrategies[i]).totalAssets();
}
```

The practical impact is denial of service on valuation-dependent operations, delayed debt servicing, and operational stress during liquidity events.

Recommendation

It's recommended that guarded calls (such as `try/catch` blocks) are used when aggregating asset values from each custom strategy, so the failure of a single strategy does not cause the entire `totalAssets` process to revert. Additionally, explicit events should be emitted whenever a strategy fails to report its asset value.

Alleviation

[Superearn, 04/13/2026]: This behavior is intentional.

We considered the alternative of having `totalAssets()` revert on strategy-level failures, but concluded that silently treating a temporarily-failing strategy's balance as zero is more dangerous than the current design.

SSA-09 | Residual ERC20 Allowance Risk In Strategy Funding Flow

Category	Severity	Location	Status
Volatile Code	● Minor	src/superearn/core/strategy/custom/CustomVault.sol (init): 298~299	● Resolved

Description

The current deposit routing uses an `approve + pull` interaction pattern, where the vault first grants allowance and the strategy subsequently pulls funds. This design increases the authorization attack surface because residual allowance can remain if the strategy does not consume exactly the approved amount or if token behavior is non-standard.

In compromised-strategy scenarios, this residual approval can be abused for unintended future `transferFrom` calls, even outside the original operation context. The risk originates from decoupling authorization and transfer settlement across external boundaries, as shown below:

```
IERC20(token).forceApprove(strategy, amount);  
ICustomStrategy(strategy).deposit(token, amount);
```

As a result, passive fund exposure increases and containment against strategy-side compromise becomes weaker.

Recommendation

It is recommended to reset the token approval to zero after the deposit operation, regardless of whether the deposit was successful or not.

Alleviation

[Superearn, 04/13/2026]: The team heeded the advice and resolved the issue by resetting residual allowance after custom strategy deposit in commit [acd69b7514fee9be6e7a8cb9ad0cf084173ead7c](#).

SSA-10 | Missing Vault Ownership Validation In Strategy Registration

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/custom/CustomVault.sol (init): 225~226	Resolved

Description

`addCustomStrategy` allows governance to register a strategy in the active accounting and operational set of `CustomVault`. The current implementation validates denomination compatibility and requires the strategy to have an initial zero-assets state. However, it does not enforce an ownership binding check such as `remoteVault == address(this)`.

Strategy execution is restricted by the `onlyRemoteVault` modifier:

```
// CustomStrategy.sol
function deposit(address token, uint256 amount) external onlyRemoteVault
nonReentrant {
    if (!allowedDepositTokens[token]) {
        revert TokenNotAllowedForDeposit(token);
    }
}
```

Since `CustomStrategy` restricts key operations such as `deposit` using the `onlyRemoteVault` modifier, any attempt to interact with such a misconfigured strategy (for example, by calling `depositToCustomStrategy()`) will result in transaction reverts. This creates a risk of operational disruption caused by registering unusable strategies due to misconfiguration.

Recommendation

It is recommended to ensure that the strategy's `remoteVault` is set to the current contract address when registering the strategy in `addCustomStrategy()`.

Alleviation

[Superearn, 04/13/2026]: The team heeded the advice and resolved the issue by verifying `strategy.remoteVault()` matches vault on registration in commit [411ad66e88f7912e950ec73b9cd6f50a67e8826c](#).

SSA-11 | Unaccounted Non-Asset Tokens Can Accumulate In CustomVault

Category	Severity	Location	Status
Volatile Code	Minor	src/superearn/core/strategy/custom/CustomVault.sol (init): 288~289, 313~314	Resolved

Description

`depositToCustomStrategy()` and `withdrawFromCustomStrategy()` allow governance or an authorized operator to move assets between `CustomVault` and a registered `CustomStrategy`, but neither function enforces that the routed token is the vault's `asset()` token. Instead, the vault accepts an arbitrary token parameter and relies on downstream strategy-side allowlists to decide whether the operation is permitted. The relevant pattern is:

```
function depositToCustomStrategy(address strategy, address token, uint256 amount)
external onlyOperator nonReentrant
function withdrawFromCustomStrategy(address strategy, address token, uint256 amount)
external
onlyOperator
nonReentrant
returns (uint256 actual)
```

This creates a mismatch between the vault's accounting model and its operational token-routing surface.

`CustomVault.totalAssets()` only accounts for idle `asset()` balance plus the USDT-denominated `totalAssets()` reported by registered strategies, so the vault is economically modeled as a single-asset system.

However, if strategy-side configuration allows non-asset tokens to be deposited into or withdrawn from a strategy, those tokens can move through the vault without any vault-level asset-type invariant. In particular, once a non-asset token is withdrawn back to the vault, it may remain outside the vault's canonical accounting path unless it is immediately converted, swept, or otherwise handled through a separate process.

As a result, asset reporting, reconciliation, and downstream assumptions about vault holdings can diverge from actual balances if non-canonical tokens are allowed to circulate through these management functions.

Recommendation

If `CustomVault` is intended to remain a strictly single-asset vault, enforce `token == asset()` in both `depositToCustomStrategy` and `withdrawFromCustomStrategy` so vault-side routing matches vault-side accounting.

Alleviation

[Superearn, 04/13/2026]: The team heeded the advice and resolved the issue by enforcing `asset()` token in custom strategy routing in commit [1aa0899eb828ab0ab9eec0d892fab1834d03a94d](#).

SSA-17 Double Subtraction Of Committed Shares Causes Denial Of Service In `premintCooldownVault`

Category	Severity	Location	Status
Logical Issue	Minor	src/superearn/core/strategy/custom/CustomYearnStrategy.sol (0416-a6fb): 258	Resolved

Description

`requestRedeem(uint256 shares)` records debt and increments `committedExternalShares`, but it does not burn the corresponding `CustomVault` shares.

`withdrawFromCustomVault(uint256 shares)` later calls `customVault.redeem(shares, address(this), address(this))`, which reduces `externalShareToken.balanceOf(address(this))` immediately, while `committedExternalShares` is only released later in `repayPredepositDebt(uint256 predepositId)`.

Because `premintCooldownVault()` computes `_available` as `_susdtBalance > committedExternalShares ? _susdtBalance - committedExternalShares : 0`, the strategy subtracts shares that are no longer in `_susdtBalance` during the gap between redemption and debt settlement. The same stale-accounting pattern applies to the analogous `shareBalance - committedExternalShares` calculation in `liquidateAllPositions()`.

Recommendation

Update the share-accounting flow so committed shares are excluded exactly once at every stage of the withdrawal lifecycle.

In particular, after `withdrawFromCustomVault()` redeems shares and reduces `externalShareToken.balanceOf(address(this))`, ensure `committedExternalShares` is reduced immediately or otherwise prevented from being subtracted again in `premintCooldownVault()` and `liquidateAllPositions()` before `repayPredepositDebt()` executes.

Use a single source of truth for share availability, and keep debt-state transitions synchronized across `requestRedeem()`, `withdrawFromCustomVault()`, and `repayPredepositDebt()`. If delayed settlement must be preserved, track redeemed-but-unsettled shares separately from still-held committed shares so only currently held shares are deducted from live balances. This should ensure availability calculations remain conservative without creating a temporary illiquidity window or avoidable shortfall reporting.

Alleviation

[Superearn, 04/21/2026]: The team heeded the advice and resolved the issue by tracking `redeemedButUnsettledShares` in commit [0242c5f357634e582c33758aea242dfbff448a03](#).

SSA-12 | Direct Aggregation Of Share And Asset Balances In Shortfall Check

Category	Severity	Location	Status
Volatile Code	● Informational	src/superearn/core/strategy/custom/CustomYearnStrategy.sol (init): 342~343	● Resolved

Description

The `repayPredepositDebt()` function in `CustomYearnStrategy`, callable only by `CoolDownVault`, checks for a shortfall by aggregating the idle USDT balance with the strategy's want token balance as follows:

```
uint256 totalStrategyHoldings = usdtBalance + want.balanceOf(address(this));
```

Currently, this direct addition is accurate because the want token represents shares in `CoolDownVault`, which maintains a strict 1:1 ratio between shares and the underlying asset (USDT).

However, this practice is non-standard since it combines an underlying token amount (USDT) with a share token amount without explicitly converting the share tokens to their equivalent asset value. Under the present design, the calculation yields the correct value, but future changes to `CoolDownVault` or deviations from the 1:1 ratio could introduce confusion or inaccuracies.

Recommendation

It is recommended to convert the strategy's `want` token balance to its equivalent USDT value using a dedicated preview or conversion function before aggregating it with the raw USDT balance.

Alleviation

[Superearn, 04/13/2026]: The team heeded the advice and resolved the issue by converting `want` shares to USDT before shortfall aggregation in commit [c7a8dfbd20f4f791f68fa06f0c9febce7404bae3](#).

SSA-15 | Initial Strategy Binding Has No Governance Recovery Path

Category	Severity	Location	Status
Coding Style	● Informational	src/superearn/core/strategy/custom/CustomVault.sol (0413-44a6): 120~121	● Resolved

Description

`CustomVault.initialize()` sets the initial `customYearnStrategy` for the vault during bootstrap, while `migrateCustomYearnStrategy` can only be called by the strategy address that is already bound.

At the same time, the `CustomYearnStrategy` constructor reads `CustomVault(_customVault).asset()`, which means the vault must already be initialized before the strategy can be deployed successfully. This creates a circular bootstrap flow in which the vault must be initialized with the correct strategy address before the final strategy instance exists. Although that address may be predicted during deployment, the contract does not provide a governance-controlled recovery path if the initial binding is wrong.

In that case, the deployed strategy cannot use the vault functions protected by `onlyCustomYearnStrategy`, and governance cannot correct the binding through a normal administrative action. As a result, an initialization mistake can leave the pair unusable until it is recovered through redeployment or another bootstrap-specific process.

Recommendation

We recommend that the team exercise particular care when setting the strategy address, as it is fixed during deployment and cannot be updated afterward. Otherwise, consider adding a governance-only function to update or correct the linked strategy address in the vault after deployment.

Alleviation

[Superearn, 04/17/2026]: The team heeded the advice and resolved the issue by decoupling strategy binding from the initialize function in commit [317c2fe5735a9a078cf226a3511db45298291125](#).

SSA-16 Recorded Cooldown Is Not Enforced Before Predeposit Debt Retrieval

Category	Severity	Location	Status
Logical Issue	● Informational	src/superearn/core/strategy/custom/CustomYearnStrategy.sol (0413-44a6): 267~272	● Resolved

Description

`predepositDebtRetrievable()` is used to determine whether a predeposit debt can be retrieved, and in `CustomYearnStrategy` it returns `true` once the strategy has enough available USDT after accounting for prior reserved debt and the shortfall branch.

The issue is that the function never checks whether the recorded cooldown period has actually elapsed. The strategy records a timestamp for each debt entry and also returns a non-zero cooldown to `CooldownVault.predeposit()`, but `predepositDebtRetrievable()` relies only on balance readiness and shortfall conditions when reporting retrievability. If the recorded cooldown is intended to be a hard timing gate for debt settlement, the current implementation does not enforce that invariant. As a result, a debt may become retrievable as soon as enough USDT is prepared, even if the recorded cooldown window has not yet passed.

Recommendation

We recommend discussing this finding with the team to confirm whether the current implementation is intended. If not, enforce the cooldown explicitly in `predepositDebtRetrievable()` by checking the cooldown snapshot associated with the specific predeposit request before returning `true`. The same timing check should also be applied in the repayment path to keep retrieval and settlement semantics aligned. If the current behavior is intentional, document clearly that the recorded cooldown is informational only and that retrievability is determined by prepared liquidity rather than elapsed time.

Alleviation

[Superearn, 04/17/2026]: The team has added documentation to clarify this in commit [f854ac2a006ec43f9aab55ca953128e64c95ab9f](#).

SSA-18 | CustomYearnStrategy Keeps A Dead Claim Path

Category	Severity	Location	Status
Inconsistency	● Informational	src/superearn/core/strategy/custom/CustomYearnStrategy.sol (init): 193	● Resolved

Description

`CustomYearnStrategy` overrides `repayPredepositDebt()` and bypasses `requestClaim()`. The custom repayment path updates `accRepaidDebtAmount`, sets `record.isClaimed = true`, and transfers USDT to `CoolDownVault` on its own.

However, `requestClaim()` only checks whether the strategy already holds enough USDT for `record.usdtAmount`, then flips `isClaimed`. It does not enforce the reserved balance logic for prior debts, it does not update `accRepaidDebtAmount`, and it does not account for partial repayment through `remainingPredepositDebt`. If a future refactor starts using `requestClaim()` because the base class suggests that pattern, that code will bypass the accounting that `CustomYearnStrategy.repayPredepositDebt()` relies on.

Recommendation

It's recommended to make the `requestClaim()` function always revert, to prevent future inconsistencies and confusion during maintenance. This ensures all claim and debt settlement flows remain centralized in the current custom repayment path, reducing the risk of accounting errors if other team members or future upgrades revert to using the outdated function.

Alleviation

[Superearn, 04/21/2026]: The team heeded the advice and resolved the issue by adding `revert` to the unused `requestClaim()` to improve code maintainability in commit [91f847f23a826820e9cdab214d0ca3fc6aef58d2](#).

APPENDIX | SUPEREARN - STRATEGY AUDIT

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Financial Manipulation	Financial Manipulation findings indicate issues in design that may lead to financial losses.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Superearn - Strategy Audit Security Assessment | CertiK Assessed on Apr 27th, 2026 | Copyright © CertiK

